

CSS

È un linguaggio indipendente con una sua sintassi. Definisce la "forma"/"l'illusrazione" del documento HTML.

Quando scriviamo un documento HTML, per "decorare" la pagina basta scrivere le regole CSS e "agganciarle" alle regole HTML

Esempio:

```
h1 { color: green; }
```

```
p { font-size: small; color: black; }
```

Abbiamo due elementi:

- Il **Selettore**, ovvero identifica l'elemento o gli elementi a cui applicare lo stile, nell'esempio h1 o p
- Il **Dichiarazione**, ovvero fornisce le indicazioni per cambiare lo stile. Si ha una coppia (proprietà:valore) separati dai :

Per applicare "forma"/"l'illusrazione" che vogliamo al documento HTML, ci sono due modi:

- collegare il file txt con estensione .css al documento HTML → inserendo all'interno del tag <head> il tag <link>

```
<!DOCTYPE html>
<html lang="it">

<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Documento</title>
<link rel = "stylesheet" type = "text/css" href = "mystyle.css">
</head>

<body>
... ..
</body>
</html>
```

- scrivere all'interno del documento HTML →

- inserendo all'interno del tag <head> il tag <style> ... </style>
- inserendo l'attributo style all'interno dei tag che si vogliono modificare, non andrebbe mai usato perché mischiano la struttura con la presentazione

- I Selettori Semplici

- **Selettore Elemento** → Seleziono tutti gli elementi di quel tipo

```
/* In questo caso tutti gli p del documento HTML avrà il colore rosso e saranno centrati */  
  
p {  
    color : red;  
    text-align: center;  
}
```

- **Selettore Classe** → Applico gli stili a tutti gli elementi di una classe.
In HTML li individuano con l'attributo class e un identificatore.
In CSS uso come selettore **.nome_classe** (identificatore preceduto dal punto)

```
<!-- In HTML -->  
  
<p class="evidenziato">Questo è un paragrafo evidenziato.</p>  
  
/* In CSS */  
  
.evidenziato {  
    color: red;  
    font-weight: bold;  
}
```

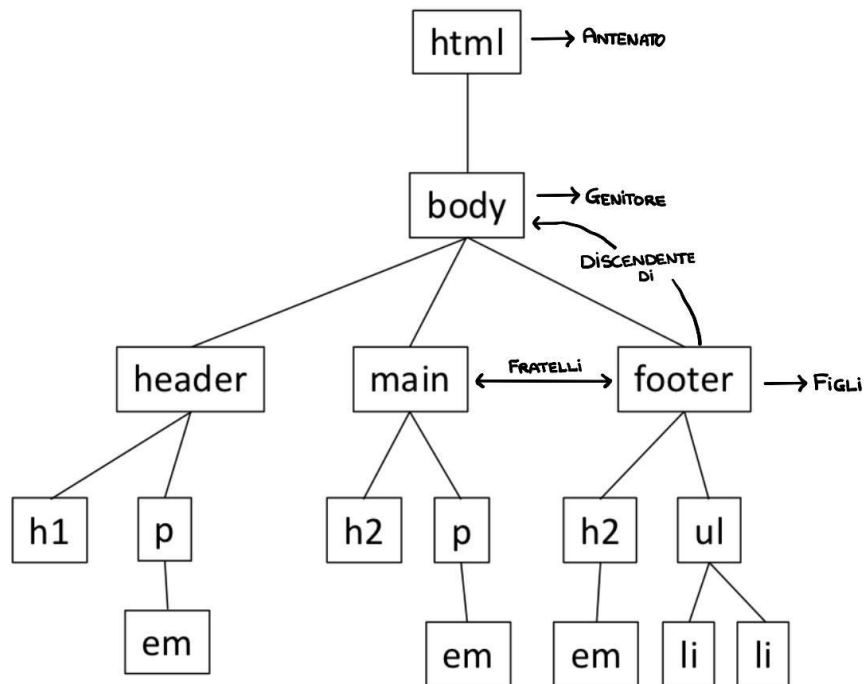
- **Selettore Id** → Seleziono l'elemento con quell'Id. In CSS scrivo l'Id preceduto dal cancelletto

```
<!-- in HTML -->  
  
<p id="titolo">Questo è un titolo speciale.</p>  
  
/* In CSS */  
  
#titolo {  
    color: blue;  
    font-size: 24px;  
    text-transform: uppercase;  
}
```

N.B: se dei selettori hanno le stesse caratteristiche di stile si possono "radunare" in un unico blocco, scrivendoli uno accanto all'altro e inserendo una virgola tra essi

- I Selettori Composti e DOM

Le relazioni tra gli elementi di una Pagina Web sono definite da una struttura ad albero.



I selettori composti servono per selezionare elementi HTML in base alla loro relazione con altri elementi nel documento HTML.

- **Selettore per Discendente** → seleziona tutti gli elementi annidati dentro un altro indipendentemente dal livello d annidamento. Si usa lo spazio

```
/* Seleziona tutti gli em che si trovano dentro un p */
```

```
p em {  
  color: green;  
}
```

- **Selettore per Figlio** → seleziona solo i figli diretti di un elemento. Si usa il >

```
/* Seleziona solo i p che sono figli diretti di < div > */
```

```
div > p {  
  background-color: yellow;  
}
```

- **Selettore per Fratello** → seleziona un elemento che segue un altro elemento al livello gerarchico.

```
/* Se un <p> segue immediatamente un <div> verrà selezionato */
```

```
div + p {  
    background-color: yellow;  
}
```

- **Selettore per Fratello** → seleziona tutti gli elementi dello stesso tipo che si trovano dopo un determinato elemento. Si usa ~

```
/* Seleziona tutti i <p> che sono fratelli successivi di <div> */
```

```
div ~ p {  
    background-color: yellow;  
}
```

Pseudo Classi

Le Pseudo Classi permettono di selezionare gli elementi in base al loro stato o posizione all'interno della pagina.

```
/* Cambia stile ai link già visitati */
```

```
a:visited {  
    color: purple;  
}
```

Ecco alcune Pseudo Classi

:root → Seleziona l'elemento radice della pagina (<html>).

:empty → Seleziona elementi HTML senza contenuto.

:only-child → Seleziona l'elemento se è l'unico figlio del genitore.

:first-child → Seleziona il primo figlio di un elemento padre.

:last-child → Seleziona l'ultimo figlio di un elemento padre.

:nth-child(n) → Seleziona il figlio n-esimo di un elemento padre.

:nth-last-child(n) → Seleziona il n-esimo figlio contando dalla fine.

:nth-of-type(n) → Seleziona il n-esimo elemento di un certo tipo.

:nth-last-of-type(n) → Seleziona il n-esimo elemento di un certo tipo, contando dalla fine.

:hover → Cambia stile quando il mouse passa sopra l'elemento.

:focus → Cambia stile quando un elemento (come un input) è attivo.

:visited → Cambia stile ai link già visitati.

Pseudo Elementi

Gli pseudo-elementi permettono di applicare stili a parti specifiche di un elemento HTML, senza dover modificare il markup.

Ecco Pseudo Elementi

::before → Inserisce contenuto prima di un elemento.

::after → Inserisce contenuto dopo un elemento.

::first-letter → Seleziona la prima lettera di un testo.

::first-line → Seleziona la prima riga di un paragrafo.

::selection → Modifica lo stile del testo selezionato.

Selettori con attributi

I selettori con attributi permettono di applicare stili agli elementi in base agli attributi HTML, e ai loro valori

Selettori di attributi:

[attribute] → Seleziona elementi che hanno l'attributo specificato.

[attribute=value] → Seleziona elementi con un valore esatto.

[attribute~=value] → Seleziona elementi con un valore presente in un elenco separato da spazi.

[attribute^=value] → Seleziona elementi il cui valore inizia con una stringa specifica.

[attribute\$=value] → Seleziona elementi il cui valore termina con una stringa specifica.

[attribute*=value] → Seleziona elementi il cui valore contiene una stringa specifica.

Ereditarietà

Alcune proprietà vengono ereditate dagli elementi discendenti. Altre proprietà non vengono ereditate automaticamente. L'eredità si basa sulla struttura del DOM, dove gli elementi possono ricevere stili dai loro genitori.

Proprietà ereditate:

- azimuth
- border-collapse
- border-spacing
- caption-side
- color
- cursor
- direction
- elevation
- empty-cells
- font-family
- font-size
- font-style
- font-variant
- font-weight
- font
- letter-spacing
- line-height
- list-style-image
- list-style-position
- list-style-type
- list-style
- orphans
- pitch-range
- pitch
- quotes
- stress
- text-align
- text-indent
- text-transform
- visibility
- voice-family
- white-space
- widows
- word-spacing

Conflitti

In CSS, i conflitti fra le varie dichiarazioni sono la normalità. Posso applicare più stili allo stesso elemento, alcune proprietà le eredito. Quando ho più dichiarazione di stile, queste vanno in conflitto e vince la seconda annullando la prima.

Se non voglio che una regola venga sovrascritta la posso dichiarare con *!important*

```
p {  
    color: green !important;  
}
```

Cascade

Il cascade è il meccanismo che i browser usano per determinare quali regole CSS applicare a un elemento quando ci sono più dichiarazioni in conflitto. Dato che gli stili possono provenire da fonti diverse, il browser deve decidere quale valore di proprietà applicare.

Gli stili possono provenire da tre fonti principali:

1. User-Agent Stylesheet (del browser) → Ogni browser ha uno stile predefinito per gli elementi HTML. Ad esempio, <h1> è più grande di <p> per impostazione predefinita.
2. Author Stylesheet (dell'autore - noi sviluppatori) → Sono i CSS scritti direttamente da chi sviluppa la pagina.
3. User Stylesheet (del lettore) → L'utente può definire i propri stili, ad esempio nei browser accessibili per chi ha problemi di vista (può aumentare il contrasto o modificare i font).

Calcolo Specificità

La specificità nei CSS è un metodo per determinare quale dichiarazione ha la priorità quando più regole si applicano allo stesso elemento. Si misura con quattro valori [a, b, c, d], e il confronto avviene in ordine di importanza da sinistra a destra. I valori sono determinati nel seguente modo:

- a. 1 se la dichiarazione è inline, 0 altrimenti
- b. numero di selettori id
- c. numero di selettori di classe, attributo o pseudo-classe
- d. numero di selettori elemento o pseudo elemento

* {}	/* a=0 b=0 c=0 d=0 -> specificity = 0,0,0,0 */
li {}	/* a=0 b=0 c=0 d=1 -> specificity = 0,0,0,1 */
li::first-line {}	/* a=0 b=0 c=0 d=2 -> specificity = 0,0,0,2 */
ul li {}	/* a=0 b=0 c=0 d=2 -> specificity = 0,0,0,2 */
ul ol+li {}	/* a=0 b=0 c=0 d=3 -> specificity = 0,0,0,3 */
h1 + *[rel=up] {}	/* a=0 b=0 c=1 d=1 -> specificity = 0,0,1,1 */
ul ol li.red {}	/* a=0 b=0 c=1 d=3 -> specificity = 0,0,1,3 */
li.red.level {}	/* a=0 b=0 c=2 d=1 -> specificity = 0,0,2,1 */
#x34y {}	/* a=0 b=1 c=0 d=0 -> specificity = 0,1,0,0 */
style=""	/* a=1 b=0 c=0 d=0 -> specificity = 1,0,0,0 */

- Proprietà relative ai font

In CSS, abbiamo proprietà relative ai font come font-family, font-size, font-weight, font-style...

- *Font Family*

Tutti i font cominciano con la lettera maiuscola, tranne i font generici. Sono separati da virgole. Se il loro nome contiene spazi vanno fra virgolette.

Se il font richiesto è disponibile nel sistema, viene usato.

Se non è disponibile, il browser cerca un'alternativa.

Se nessuna alternativa è disponibile, viene usato un font di sistema generico.

Non tutti i font sono disponibili, dipendono dal sistema operativo e sono installati nel pc.

Possiamo scaricare un font e per aggiungerlo al progetto incorporiamo il font nel foglio di stile.

Usa il font altrove nel foglio di stile.

```
@font-face {  
  font-family: 'Roboto';  
  src: url('Roboto-Light-webfont.woff') format('woff');  
}  
  
p {  
  'Roboto', sans-serif;  
}
```

- *Font Size*

Specifica la dimensione del carattere, ad esempio `h1 { font-size: 150%; }`. Valori ammessi sono:

- unità di lunghezza, percentuale
- xx-small | x-small | small | medium | large | x-large | xx-large | smaller | larger

- *Unità di misura*

Unità relative (dipendono dal contesto)

% (percentuale) → Indica una dimensione relativa rispetto al valore dell'elemento padre.

em → Misura relativa alla dimensione del font dell'elemento padre.

rem (root em) → Simile a em, ma relativa alla dimensione del font dell'elemento radice (html).

ch → Larghezza di un carattere "0" nella font attuale, utile per gestire testi.

vw (viewport width) → 1% della larghezza della finestra del browser.

vh (viewport height) → 1% dell'altezza della finestra del browser.

vmin (viewport minimum) → Il valore minore tra vw e vh, utile per adattamenti reattivi.

Unità assolute (misure fisse e indipendenti dal contesto)

px (pixel) → Unità fissa, corrisponde a **1/96 di pollice** in CSS3.

pt (punto) → 1/72 di pollice, usato soprattutto nella stampa.

pc (pica) → Unità tipografica dove **1 pica = 12 punti (pt)**.

mm (millimetri) → Misura basata sul sistema metrico, usata nella stampa.

cm (centimetri) → 1 cm = 10 mm, più comune nei documenti stampabili.

in (inch/pollice) → Misura fisica reale, 1 pollice = **2.54 cm**.

- **Line Height**

Indica il valore dell'altezza del rigo. I valori ammessi sono:

- number | length measurement | percentage |
- normal

- Proprietà relative al testo

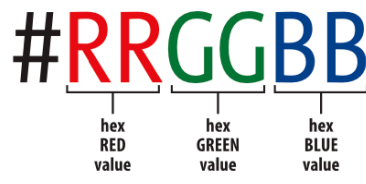
In CSS, abbiamo proprietà relative al testo come color, line-height, text-decoration, text-align, ...

- **Line Height**

Il colore del testo viene ereditato dagli elementi, non influenza solo il testo.

```
p {  
    color: yellow;  
}
```

```
h1 {  
    color: #666666;  
}
```



Con RGBA indichiamo il livello di trasparenza del colore di un testo.

```
h1 {  
    color: rgba ( 0, 0, 0, 1);  
}
```

- **Text Align**

Indica la posizione di un paragrafo. I valori ammessi sono:

- left | right | center | justify

text-align: left

Paragraph 1. The text-align property controls the horizontal alignment of the text within an element. It does not affect the alignment of the element on the page. The resulting text behavior of the various values should be fairly intuitive.

text-align: right

Paragraph 2. The text-align property controls the horizontal alignment of the text within an element. It does not affect the alignment of the element on the page. The resulting text behavior of the various values should be fairly intuitive.

text-align: center

Paragraph 3. The text-align property controls the horizontal alignment of the text within an element. It does not affect the alignment of the element on the page. The resulting text behavior of the various values should be fairly intuitive.

text-align: justify

Paragraph 4. The text-align property controls the horizontal alignment of the text within an element. It does not affect the alignment of the element on the page. The resulting text behavior of the various values should be fairly intuitive.

- **Text Decoration**

Indica la decorazione del testo. I valori ammessi sono:

- none | underline | overline | line-through | blink

I've got laser eyes.

text-decoration: underline

I've got laser eyes.

text-decoration: overline

~~I've got laser eyes.~~

text-decoration: line-through

- **Text Transform**

Cambia le maiuscole e le minuscole. I valori ammessi sono:

- None | capitalize | lowercase | uppercase

And I know what you're thinking.

text-transform: none (as was typed in)

And I Know What You'Re Thinking.

text-transform: capitalize

and i know what you're thinking.

text-transform: lowercase

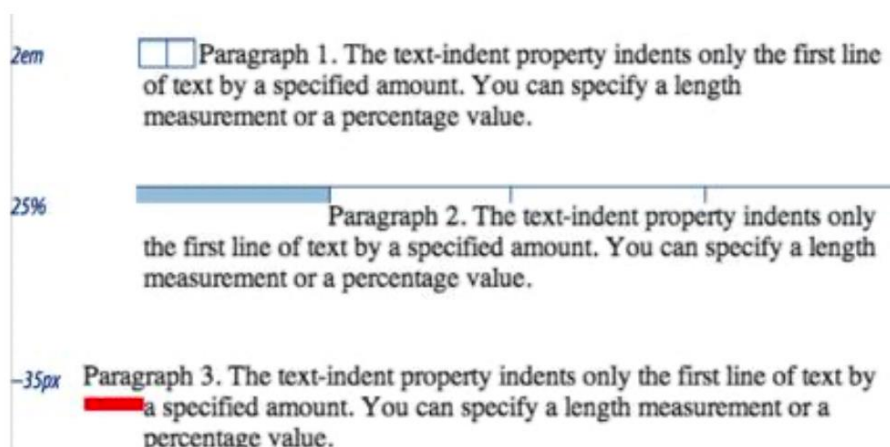
AND I KNOW WHAT YOU'RE THINKING.

text-transform: uppercase

- **Text Indent**

Cambia l'indentazione del primo rigo. I valori ammessi sono:

- misura di lunghezza
- percentuali



- **Text Shadow**

Inserisce l'ombra sul testo. I valori ammessi sono:

- 'horizontal offset' 'vertical offset' 'blur radius' 'color'

The Jenville Show

`text-shadow: .2em .2em silver`

The Jenville Show

`text-shadow: -.3em -.3em silver;`

The Jenville Show

`text-shadow: .2em .2em .05 em silver`

The Jenville Show

`text-shadow: .2em .2em .15 em silver`

The Jenville Show

`text-shadow: .2em .2em .3 em silver`

- **Ombre Multiple**

Multiple shadows

The Jenville Show

`text-shadow: -.7em -.5em .2em silver,
.2em .2em .1em gray;`

Outer glow

The Jenville Show

`text-shadow: 0 0 .7em purple;`

- **Letter Spacing e Word Spacing**

Inserisce spazi tra parole o lettere. I valori ammessi sono:

- misura di lunghezza | normal

letter-spacing: 8px;

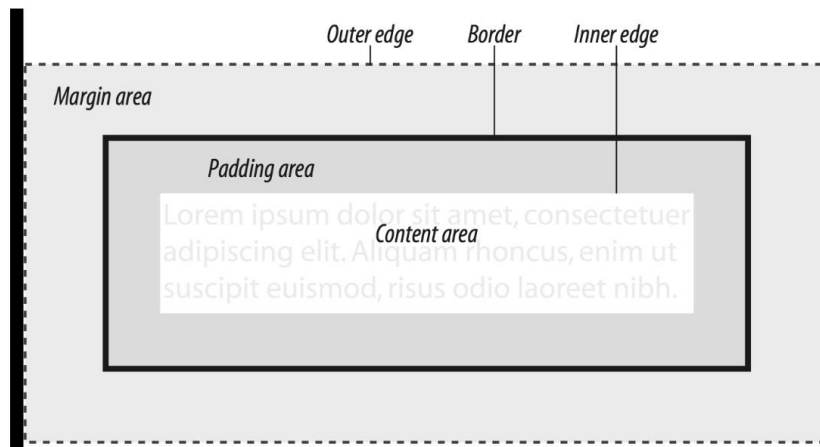
B l a c k G o o s e B i s t r o S u m m e r M e n u

word-spacing: 1.5em;

Black Goose Bistro Summer Menu

• BOX MODEL

Ogni elemento html è contenuto in un box rettangolare. Ogni box ha delle proprietà CSS applicate. In ogni box possiamo visualizzare un box, con la proprietà border.



• Dimensione – Width & Height

Con Width possiamo cambiare il "peso" ovvero la dimensione del box. I valori ammessi sono:

- Misura della Lunghezza | Percentuale | auto | inherit (ereditata)

Con Height possiamo cambiare l'altezza ovvero la dimensione del box. I valori ammessi sono:

- Misura della Lunghezza | Percentuale | auto | inherit (ereditata)

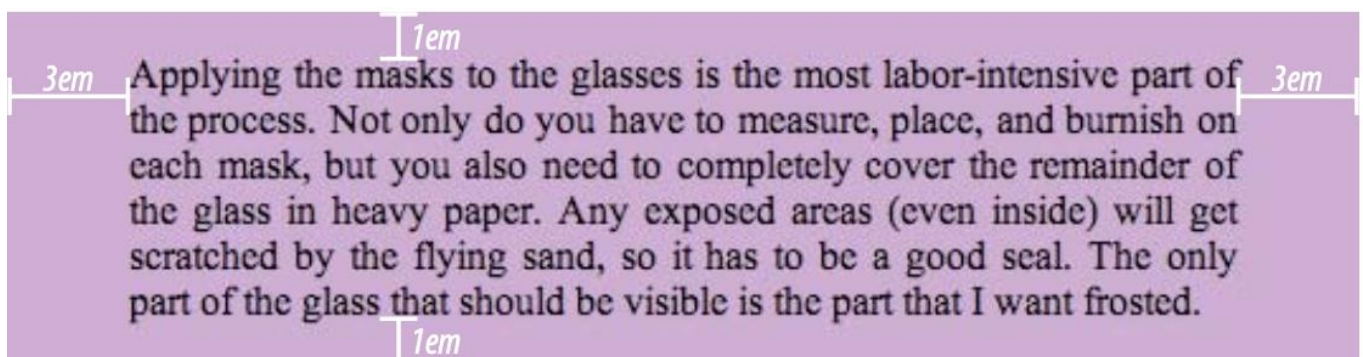
• Padding

Il Padding è la parte più interna del box model, ovvero lo spazio attorno al contenuto di un elemento, all'interno di margini o bordi definiti. I valori ammessi sono:

- Misura della Lunghezza | Percentuale | inherit (ereditata)

Possiamo definire delle categorie:

- PADDING – TOP → Misura della Lunghezza | Percentuale | inherit (ereditata)
- PADDING – RIGHT → Misura della Lunghezza | Percentuale | inherit (ereditata)
- PADDING – BOTTOM → Misura della Lunghezza | Percentuale | inherit (ereditata)
- PADDING – LEFT → Misura della Lunghezza | Percentuale | inherit (ereditata)



- **Margine del box**

I valori ammessi per il margine sono:

- Misura della Lunghezza | Percentuale | auto | inherit (ereditata)

Possiamo definire delle categorie:

- MARGIN – TOP → Misura della Lunghezza | Percentuale | auto | inherit (ereditata)
- MARGIN – RIGHT → Misura della Lunghezza | Percentuale | auto | inherit (ereditata)
- MARGIN – BOTTOM → Misura della Lunghezza | Percentuale | auto | inherit (ereditata)
- MARGIN – LEFT → Misura della Lunghezza | Percentuale | auto | inherit (ereditata)

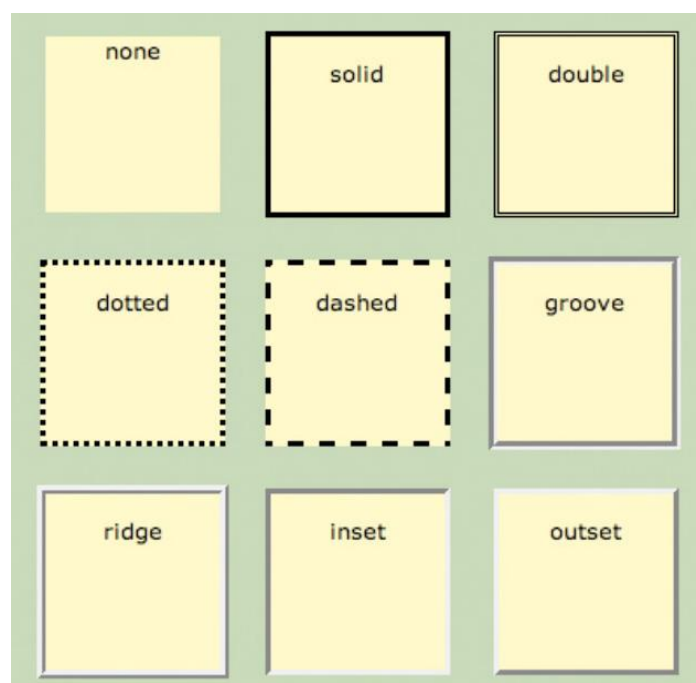
- **Border**

Del contorno di un elemento possiamo specificare:

- ***Border – style*** → none | dotted | dashed | solid | double | groove | ridge | inset | outset | inherit.

Possiamo definire delle categorie:

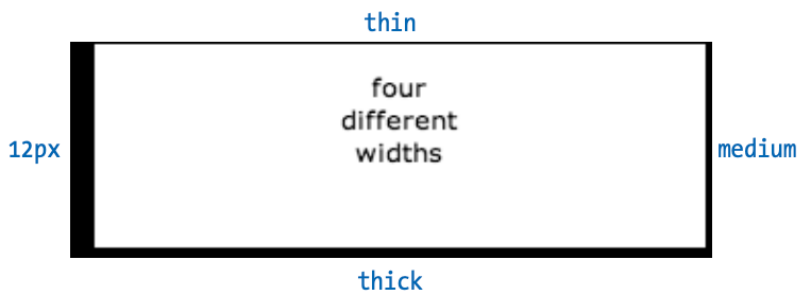
- BORDER – TOP – STYLE → none | dotted | dashed | solid | double | groove | ridge | inset | outset | inherit
- BORDER – RIGHT – STYLE → none | dotted | dashed | solid | double | groove | ridge | inset | outset | inherit
- BORDER – BOTTOM – STYLE → none | dotted | dashed | solid | double | groove | ridge | inset | outset | inherit
- BORDER – LEFT – STYLE → none | dotted | dashed | solid | double | groove | ridge | inset | outset | inherit



- ***Border – width*** → Unità di lunghezza | thin (sottile) | sottile | medio | thick | inherit (ereditata)

Possiamo definire delle categorie:

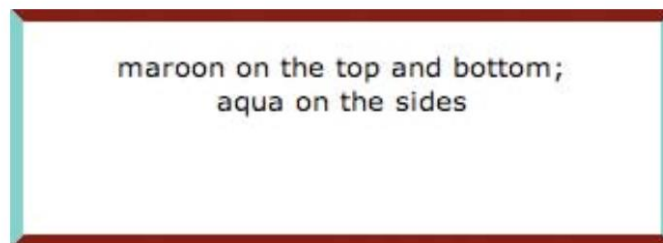
- **BORDER – TOP – WIDTH** → Unità di lunghezza | thin (sottile) | medium (medio) | thick | inherit (ereditata)
- **BORDER – RIGHT – WIDTH** → Unità di lunghezza | thin (sottile) | medium (medio) | thick | inherit (ereditata)
- **BORDER – BOTTOM – WIDTH** → Unità di lunghezza | thin (sottile) | medium (medio) | thick | inherit (ereditata)
- **BORDER – LEFT – WIDTH** → Unità di lunghezza | thin (sottile) | medium (medio) | thick | inherit (ereditata)



- ***Border – color*** → Nome Colore o valore RGB | trasparente | inherit (ereditata)

Possiamo definire delle categorie:

- **BORDER – TOP – COLOR** → Nome Colore o valore RGB | trasparente | inherit (ereditata)
- **BORDER – RIGHT – COLOR** → Nome Colore o valore RGB | trasparente | inherit (ereditata)
- **BORDER – BOTTOM – COLOR** → Nome Colore o valore RGB | trasparente | inherit (ereditata)
- **BORDER – LEFT – COLOR** → Nome Colore o valore RGB | trasparente | inherit (ereditata)



- ***Border*** → border - style border - width border - color | inherit (ereditata)

Possiamo definire delle categorie:

- **BORDER – TOP** → border - style border - width border - color | inherit (ereditata)
- **BORDER – RIGHT** → border - style border - width border - color | inherit (ereditata)
- **BORDER – BOTTOM** → border - style border - width border - color | inherit (ereditata)
- **BORDER – LEFT** → border - style border - width border - color | inherit (ereditata)

- **Box Sizing**

Definisce la grandezza del box. I valori ammessi per il margine sono:

- Content – Box → include nella grandezza padding e border
- Border – Box → Include nella grandezza solo il contenuto

- **Margini Inline**

Gli elementi inline sono quegli elementi che non iniziano su una nuova riga e occupano solo lo spazio necessario al loro contenuto (es. `<a>`, ecc...). Quando si parla di margini top e bottom (cioè i margini superiori e inferiori), questi margini vengono ignorati dagli elementi inline, non avrà alcun effetto sul layout.

L'eccezione a questa regola riguarda gli elementi replaced, cioè elementi che sono sostituiti dal loro contenuto (per esempio, le `<img>`). Per questi elementi, i margini top e bottom vengono applicati correttamente.

- **Margini Inline**

Quando si hanno due elementi (o blocchi), uno sopra l'altro. Ogni elemento ha un margine superiore e inferiore, che crea dello spazio tra di loro.

Ad esempio, supponiamo che:

- Il primo elemento ha un margine inferiore di 30px.
- Il secondo elemento ha un margine superiore di 20px.

Ci si aspetta che ci sia 50px di spazio tra i due elementi (30px + 20px). Però, invece di sommare i margini, si prende il margine più grande fra i due. Quindi, in questo caso, lo spazio tra i due elementi sarà di 30px.

Il collasso dei margini succede in due casi:

- Quando due elementi sono vicini tra loro.
- Quando non ci sono altri elementi che li separano.

Non collassano se l'elemento è float o absolute...???

Si può impostare un valore negativo al margine

- **Overflow**

La proprietà overflow specifica se ritagliare il contenuto o aggiungere barre di scorrimento quando il contenuto di un elemento è troppo grande per adattarsi all'area specificata. I valori ammessi per il margine sono:

- visible | hidden | scroll | auto | inherit

visible

Applying the masks to the glasses is the most labor-intensive part of the process. Not only do you have to measure, place, and burnish on each mask, but you also need to completely cover the remainder of the glass in heavy paper. Any exposed areas (even inside) will get scratched by the flying sand, so it has to be a good seal.

hidden

Applying the masks to the glasses is the most labor-intensive part of the process. Not only do you have to measure, place, and burnish on each mask, but you also need to completely

scroll

Applying the masks to the glasses is the most labor-intensive part of the process. Not only do you have to measure, place, and burnish on each mask, but you also

auto (short text)

Applying the masks to the glasses is the most labor-intensive part of the process.

auto (long text)

Applying the masks to the glasses is the most labor-intensive part of the process. Not only do you have to measure, place, and burnish on each mask, but you also

- **BACKGROUND**

Indica l'area dietro il contenuto di un elemento, che può essere un colore, un'immagine o entrambi.

- **Colore del Background**

Può essere indicato con i seguenti valori:

- Nome Colore | Trasparent | inherit (ereditata)

Del colore del background può essere espressa pure l'**opacità**, con la proprietà *opacity*, attraverso un valore che va da 0 ad 1.

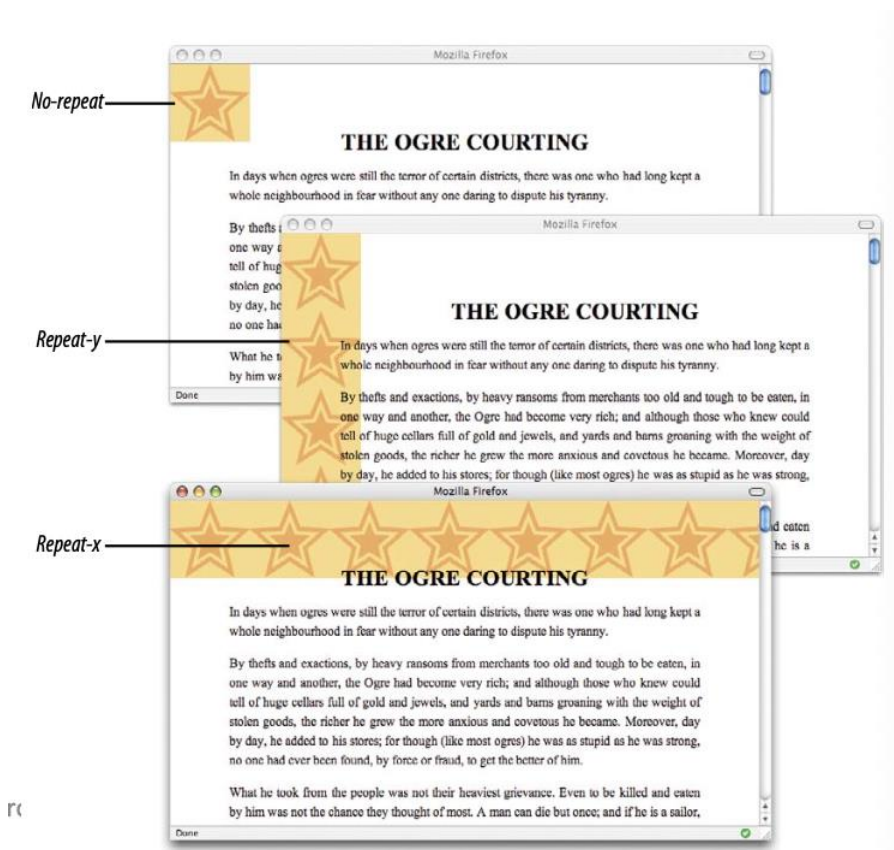
- **Immagini per Background**

Può essere indicato con i seguenti valori:

- url immagine | none | inherit (ereditata)

È possibile inserire la stessa immagine ripetuta più volte, con la proprietà *background-repeat*. I valori ammessi sono:

- Repeat | Repeat – x | Repeat – y | no – repeat | inherit



- **Posizione del Background**

Può essere indicato con i seguenti valori:

- Misura di lunghezza | Percentuale | left | center | right | top | bottom | inherit

- **Gestione dello scroll**

È definito dalla proprietà background-attachment. Può essere indicato con i seguenti valori:

- scroll | fixed | local | inherit (ereditato)

- **Cambiare il Display**

La proprietà “display” definisce il comportamento di un elemento all’interno del flusso del documento. I valori ammessi sono:

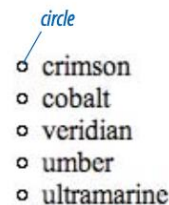
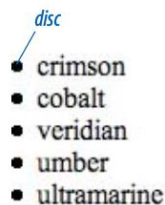
Inline | block | list-item | inline – block | table | inline – table | table – row – group | table –header – group | table –footer – group | table – row | table – column – group | table – column | table-cell | table-caption | none

Le seguenti sono nuove in CSS3 → run – in | compact | ruby | ruby – base | ruby – text | ruby – base – container | ruby – text – container

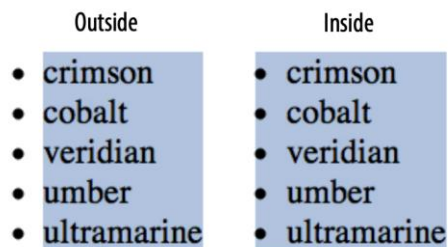
• Stili delle Liste

Ogni lista è formata da:

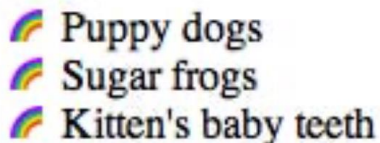
- Tipo di lista → corrisponde a definire la tipologia di indicatore dell'elemento di una lista. Si indica con la dicitura *list – style – type*. I valori ammessi sono:
 - None | disc | circle | square | decimal | decimal – leading – zero | lower – alpha | upper – alpha | lower – latin | upper – latin | lower – roman | upper – roman | lower – greek | inherit (ereditata)



- Posizione Marker → Si indica con la dicitura *list – style – position*. I valori ammessi sono:
 - inside | outside | inherit (ereditata)



- Immagini per Marker → Si indica con la dicitura *list – style – image*. I valori ammessi sono:
 - url | none | inherit (ereditata)



- **Punti specifici in una pagina**

Ogni elemento con un attributo id può diventare un'ancora, cioè un punto a cui si può fare riferimento con un link. Per creare un link verso un'ancora nella stessa pagina, si usa il simbolo # seguito dal nome dell'id.

```
<! -- Cliccando su "H", si va direttamente al punto della pagina con id="startH" -->
```

```
<p>...F | G | <a href="#startH">H</a> | I | J...</p>
```

- **Punti specifici in altre pagine**

Puoi anche collegarti a un punto specifico in un'altra pagina, sempre usando #id.

```
<a href="glossary.html#startH">See the Glossary, letter H</a>
```

```
<a href="http://www.example.com/glossary.html#startH">See the Glossary, letter H</a>
```

- **Attributo Target**

L'attributo target dice dove aprire il link. Solitamente si usa l'attributo target = "_blank", il quale apre il link in una nuova finestra o scheda.

```
<a href="http://www.oreilly.com" target="_blank">O'Reilly</a>
```

- **Altri tipi di link**

Si possono aggiungere link che fanno riferimento ad email e numeri telefonici

```
<a href="mailto:pierpaolo.loreti@uniroma2.it">Pierpaolo Loreti</a>
```

```
<a href="tel:+390672597440">Pierpaolo Loreti</a>
```

- **Stile dei link**

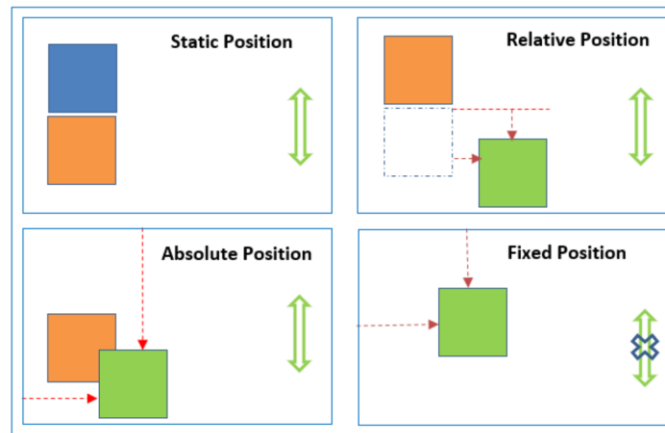
- a:link → Link non visitati
- a:visited → Link già visitati (memorizzati nella cache del browser)
- a:focus → Quando il link è selezionato (es. con TAB da tastiera)
- a:hover → Quando il mouse passa sopra (hover)
- a:active → Quando il link è attivamente cliccato (es. nel momento del click)

```
a {  
  text-decoration: none; /* Rimuove la sottolineatura da tutti i link */  
}  
  
a:link {  
  color: maroon; /* Colore dei link non visitati */  
}  
  
a:visited {  
  color: gray; /* Colore dei link già visitati */  
}  
  
a:focus {  
  color: maroon;  
  background-color: #ffd9d9;  
}  
  
a:hover {  
  color: maroon;  
  background-color: #ffd9d9;  
}  
  
a:active {  
  color: red;  
  background-color: #ffd9d9;  
}
```

• Proprietà Position

Ad ogni elemento possiamo applicare la proprietà *position*. Questa ammette i seguenti valori:

- *static* → posizione normale
- *relative* → posizionabile rispetto al suo posto
- *absolute* → posizionabile rispetto al suo contenitore
- *fixed* → posizionabile rispetto al viewport
- *inherit* → ereditata



Per specificare la posizione possiamo usare pure:

- *TOP* → misura di lunghezza | percentuale | *auto* | *inherit* (ereditato)
- *RIGHT* → misura di lunghezza | percentuale | *auto* | *inherit* (ereditato)
- *BOTTOM* → misura di lunghezza | percentuale | *auto* | *inherit* (ereditato)
- *LEFT* → misura di lunghezza | percentuale | *auto* | *inherit* (ereditato)

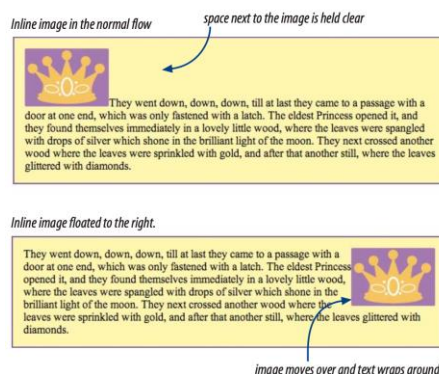
• Gestire le Sovrapposizioni

Ogni elemento può posizionarsi sopra altri. Le sovrapposizioni vengono gestite con la proprietà *z-index*. Questa ammette i seguenti valori:

- Numeri | *auto* | *inherit* (ereditato)

Per muovere un elemento tutto a destra o tutto a sinistra, permettendo agli altri elementi di circondarlo, uso la proprietà *float*. Questa ammette i seguenti valori:

- *Left* | *right* | *none* | *inherit* (ereditato)



• Caratteristiche degli Elementi Flottanti

Gli elementi flottanti (float) presentano le seguenti caratteristiche:

- Si staccano dal flusso normale ma continuano a influenzare il contenuto dei blocchi circostanti.
- Rimangono contenuti nell'area del contenuto dell'elemento che li contiene.
- I margini definiti per l'elemento flottante vengono mantenuti.

Queste caratteristiche rendono la proprietà float particolarmente utile per creare layout complessi, come colonne o sezioni affiancate.

• Clear dei Float

La proprietà *clear* viene utilizzata per evitare che gli elementi successivi si dispongano accanto a un elemento flottante. È particolarmente utile per ripristinare il flusso normale del layout. Viene espressa con i seguenti valori:

- Left | right | both (entrambi) | none (nessuno) | inherit (ereditata)

• Contenere i float

Quando un elemento contiene elementi flottanti, il contenitore potrebbe non adattarsi automaticamente alla loro altezza. Per risolvere questo problema, è necessario utilizzare la proprietà *overflow*. Viene espressa con i seguenti valori:

- Visible → mostra il contenuto che supera i limiti
- Hidden → lo ritaglia
- Scroll → aggiunge barre di scorrimento
- Auto → decide se aggiungere le barre di scorrimento automaticamente.



Hand-stitched iPhone motif
[More info](#)

L'elemento contenitore non si allunga per contenere il float



Hand-stitched iPhone motif
[More info](#)

• Contenere i float

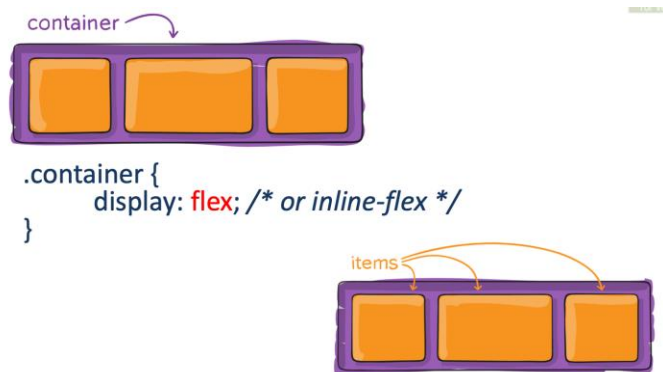
Flexbox, o Flexible Box Layout, è un modello di layout CSS progettato per distribuire lo spazio tra gli elementi di un contenitore e per allinearli in modo potente e prevedibile, anche quando le dimensioni degli elementi sono dinamiche o sconosciute. È particolarmente utile per creare layout reattivi.

• Contenere i float

Flexbox, o Flexible Box Layout, è un modello di layout CSS progettato per distribuire lo spazio tra gli elementi di un contenitore e per allinearli in modo potente e prevedibile, anche quando le dimensioni degli elementi sono dinamiche o sconosciute. È particolarmente utile per creare layout reattivi.

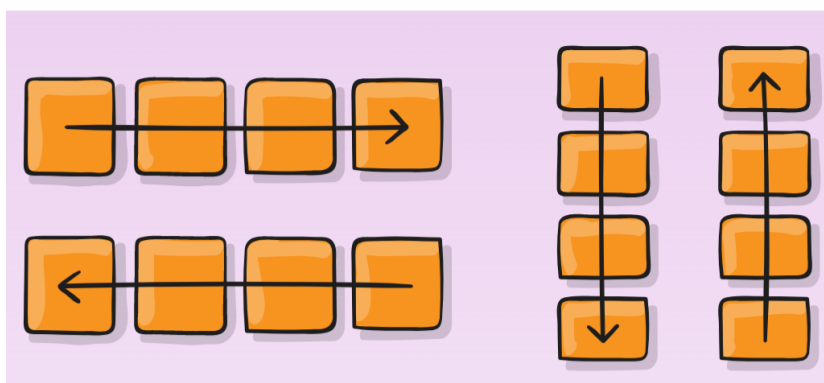
Per utilizzare Flexbox, basta definire un contenitore flex container e applicare la proprietà `display`. Questa proprietà ammette i seguenti valori:

- `flex` | `inline – flex`



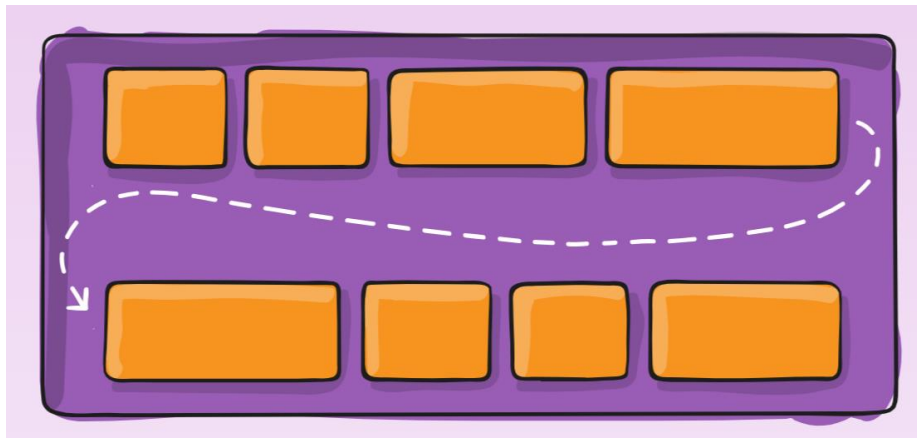
Possiamo usare la proprietà `flex – direction` che controlla la direzione in cui gli elementi vengono posizionati. Questa proprietà ammette i seguenti valori:

- `row` → gli elementi sono disposti orizzontalmente da sinistra a destra
- `row – reverse` → gli elementi sono disposti orizzontalmente da destra a sinistra
- `column` → gli elementi sono disposti verticalmente dall'alto verso il basso
- `column – reverse` → gli elementi sono disposti verticalmente dal basso verso l'alto



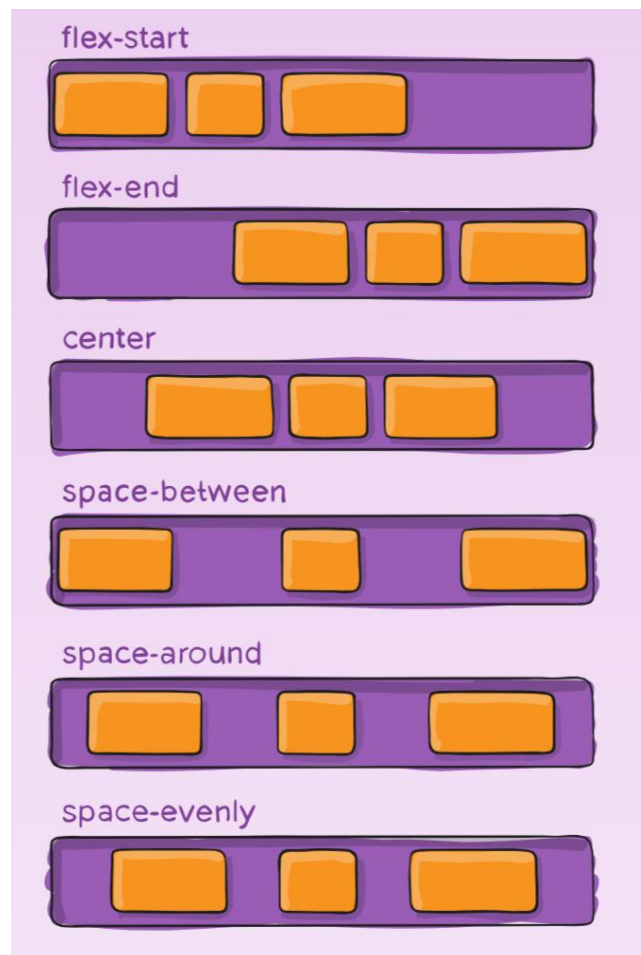
Possiamo usare la proprietà `flex – wrap` che specifica se gli elementi all'interno di un contenitore devono essere posizionati in una singola riga o su più righe. Questa proprietà ammette i seguenti valori:

- `nowrap` → gli elementi sono disposti solo su una riga
- `wrap` → gli elementi sono disposti su più righe
- `wrap – reverse` → gli elementi sono disposti su più righe ma sono invertite



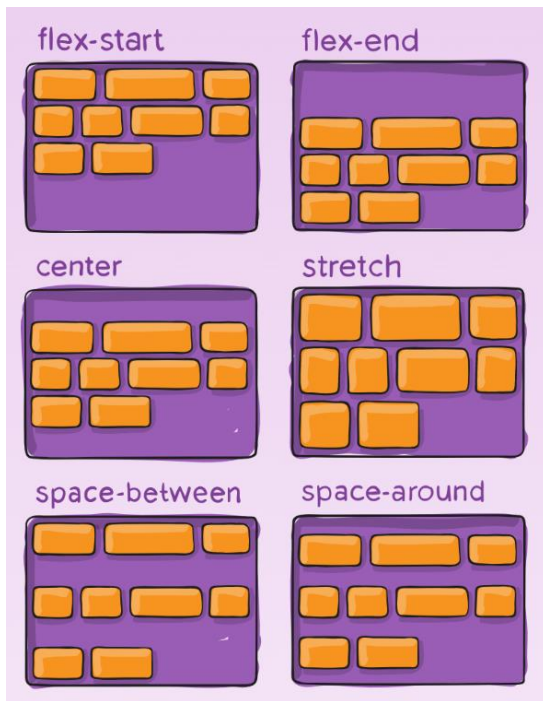
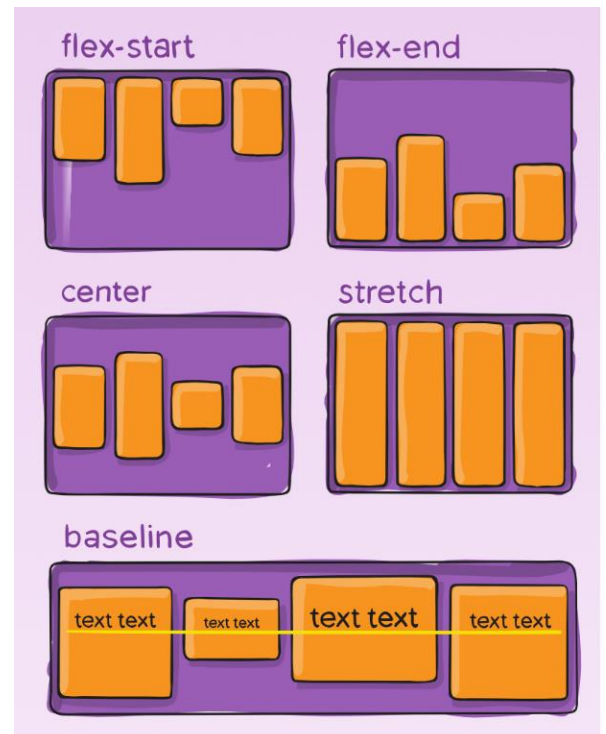
Possiamo usare la proprietà *justify-content* che controlla l'allineamento degli elementi all'intero del contenitore. Questa proprietà ammette i seguenti valori:

- *flex-start* → gli elementi sono allineati all'inizio dell'asse principale
- *flex-end* → gli elementi sono allineati alla fine dell'asse principale
- *center* → gli elementi sono centrati lungo l'asse principale
- *space-between* → lo spazio è distribuito uniformemente tra gli elementi
- *space-around* → lo spazio è distribuito uniformemente tra gli elementi evidenziando i margini
- *space-evenly* → lo spazio è distribuito uniformemente fra e intorno agli elementi



Possiamo usare la proprietà *align-items* che controlla l'allineamento degli elementi all'interno del contenitore. Questa proprietà ammette i seguenti valori:

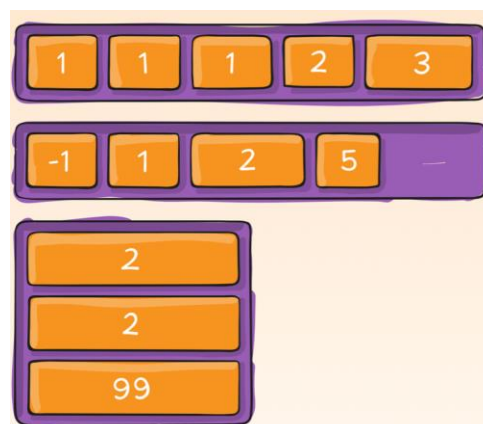
- *flex-start* → gli elementi sono allineati all'inizio dell'asse trasversale
- *flex-end* → gli elementi sono allineati alla fine dell'asse trasversale
- *center* → gli elementi sono centrati lungo l'asse trasversale
- *stretch* → gli elementi si espandono per riempire lo spazio disponibile
- *baseline* → gli elementi sono allineati alla loro linea base



Possiamo usare la proprietà *align-content* che controlla l'allineamento delle righe quando ci sono più righe di elementi. Questa proprietà ammette i seguenti valori:

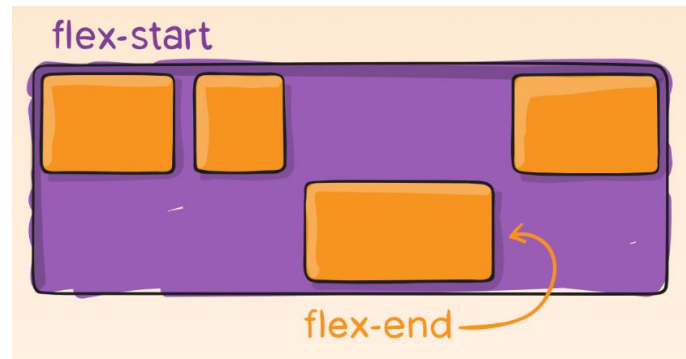
- *flex-start* → le righe sono allineate all'inizio del contenitore
- *flex-end* → le righe sono allineate alla fine del contenitore
- *center* → le righe sono centrate verticalmente
- *space-between* → le righe si espandono per riempire lo spazio disponibile
- *space-around* → lo spazio è distribuito uniformemente tra le righe
- *space-evenly* → lo spazio è distribuito uniformemente con margini ai lati delle righe

Possiamo usare la proprietà *order* che consente di specificare l'ordine degli elementi all'interno del contenitore FlexBox. Questa proprietà ammette sia valori positivi, che negativi a secondo l'ordine di visualizzazione che si vuole ottenere.



Possiamo usare la proprietà *align-self* che consente di sovrascrivere l'allineamento dato dalla proprietà *align-items* per un singolo elemento. Questa proprietà ammette i seguenti valori:

- *auto* → utilizza il valore di *align-items* del contenitore
- *flex-start* → allinea l'elemento all'inizio dell'asse trasversale
- *flex-end* → allinea l'elemento alla fine dell'asse trasversale
- *center* → centra l'elemento lungo l'asse trasversale
- *stretch* → espande l'elemento per riempire lo spazio disponibile
- *baseline* → allinea l'elemento alla linea base



- **Menù di Navigazione**

I menù di navigazione sono un elemento fondamentale per la creazione di interfacce utente. Possono essere disposti in modo verticale o orizzontale. Solitamente vengono creati applicando uno stile ad una lista HTML (`< ul >`).

serif

sans-serif

script

display

dingbats

• Tipi di Layout delle pagine web

Una pagina web può avere due tipologie di layout:

Layout Fluido → La larghezza degli elementi è proporzionale alla larghezza della finestra del browser.

- Vantaggi:

- Si adatta naturalmente ai browser.
- Evita spazi vuoti.
- Non crea scrollbar orizzontali.

- Svantaggi:

- Su schermi grandi, il testo può diventare difficile da leggere.
- Difficile prevedere l'effetto grafico finale.

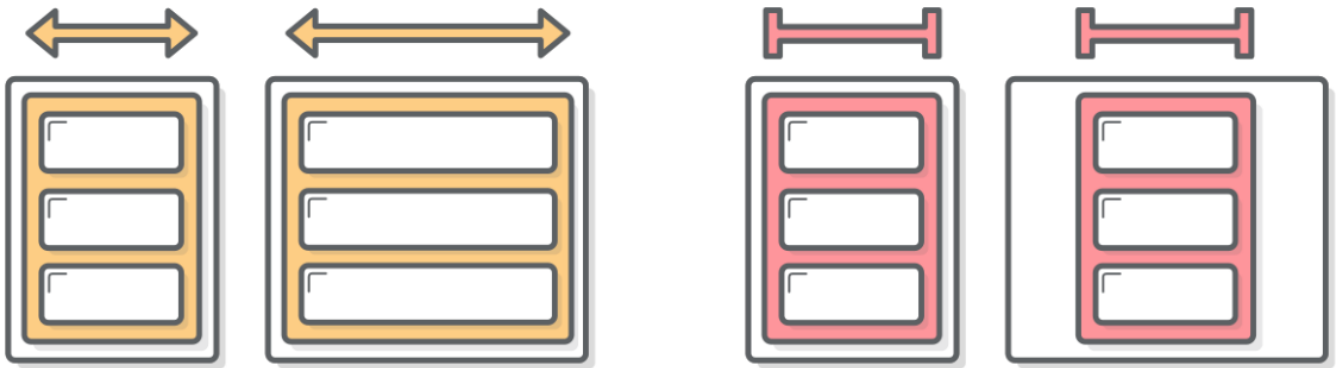
Layout Fisso → La larghezza è definita in pixel, indipendente dalla finestra del browser.

- Vantaggi:

- Più controllo sull'impaginazione.
- Facile da implementare.

- Svantaggi:

- Non si adatta bene a schermi piccoli o grandi.
- Potrebbe tagliare contenuti o creare margini inutili.



Grid

CSS Grid è un sistema di layout che consente di creare griglie complesse senza dover ricorrere a float, margin complicati o JavaScript. Ogni elemento può essere disposto in righe e colonne, decidendo:

- Quante colonne e righe ci sono
- Quanto spazio occupa ogni cella
- Dove va ogni elemento all'interno della griglia

Si basa sempre su due livelli:

1. Il contenitore: a cui si applica `display: grid` o `display: inline-grid`.
2. Gli elementi figli, che si collocano in posizioni precise.

Proprietà del contenitore Grid

Proprietà	Descrizione
<code>display</code>	Attiva il layout Grid. Usa <code>grid</code> o <code>inline-grid</code> .
<code>grid-template-columns</code>	Definisce quante colonne ha la griglia e la larghezza di ciascuna . Es: <code>1fr 2fr 1fr, repeat(3, 1fr)</code> .
<code>grid-template-rows</code>	Come sopra, ma per le righe . Es: <code>100px auto 1fr</code> .
<code>grid-template-areas</code>	Assegna nomi visivi alle aree della griglia. Serve per layout leggibili e semantici.
<code>grid-template</code>	Shorthand per definire righe, colonne e aree in una sola proprietà.
<code>grid-auto-rows</code>	Definisce la dimensione automatica delle righe che non sono dichiarate.
<code>grid-auto-columns</code>	Come sopra, ma per colonne automatiche.
<code>grid-auto-flow</code>	Imposta il flusso automatico degli elementi: <code>row</code> , <code>column</code> , <code>row dense</code> , <code>column dense</code> .
<code>gap</code>	Spazio tra righe e colonne. Valore singolo o doppio. Es: <code>gap: 10px 20px;</code> .
<code>row-gap</code> , <code>column-gap</code>	Permettono di specificare separatamente il gap verticale e orizzontale.
<code>justify-items</code>	Allinea gli elementi nelle celle lungo l'asse orizzontale. Es: <code>start</code> , <code>end</code> , <code>center</code> , <code>stretch</code> .
<code>align-items</code>	Allinea gli elementi nelle celle verticalmente.
<code>justify-content</code>	Allinea l'intera griglia nel contenitore, orizzontalmente.
<code>align-content</code>	Allinea l'intera griglia nel contenitore, verticalmente.

Proprietà degli elementi figli

Proprietà	Descrizione
grid-column-start	Inizia alla colonna X . Es: grid-column-start: 2.
grid-column-end	Finisce alla colonna Y . Es: grid-column-end: 4.
grid-row-start	Inizia alla riga X .
grid-row-end	Finisce alla riga Y .
grid-column	Shorthand: start / end. Es: grid-column: 2 / 4.
grid-row	Come sopra, ma per le righe.
grid-area	Può essere il nome di un'area dichiarata in grid-template-areas, oppure 4 numeri: row-start / col-start / row-end / col-end.
justify-self	Allinea il singolo elemento orizzontalmente nella sua cella.
align-self	Allinea il singolo elemento verticalmente nella cella.
place-self	Shorthand di align-self e justify-self. Es: place-self: center end.

Responsive

Per pagine responsive si intende la creazione di layout che si adatta in automatico alla dimensione dello schermo. Ci sono varie tecniche per la creazione di layout responsive.

1. Viewport meta tag

Questo tag va sempre inserito nell'**head** di ogni documento HTML per dire al browser di adattarsi alla larghezza del dispositivo.

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

2. Media Queries

Le media query sono delle regole CSS che vengono applicate solo quando certe condizioni sono vere, come ad esempio la larghezza dello schermo.

```
@media (max-width: 768px) {  
  .contenitore {  
    flex-direction: column;  
  }  
}
```

Questa regola viene applicata solo se la larghezza dello schermo è 768px o meno. Il CSS responsive si può scrivere in due modi: Mobile First o Desktop First.

Mobile First

L'idea è quella di scrivere il CSS per gli schermi piccoli, poi espandere mediante media query con `min-width` per schermi più grandi.

```
/* Base: mobile */
body {
    font-size: 16px;
    display: flex;
    flex-direction: column;
}

/* Tablet e oltre */
@media (min-width: 768px) {
    body {
        flex-direction: row;
    }
}

/* Desktop largo */
@media (min-width: 1200px) {
    body {
        max-width: 1200px;
        margin: 0 auto;
    }
}
```

Desktop First

L'idea invece è quella di scrivere il CSS per schermi grandi, poi adattarlo a schermi più piccoli con le media query con `max-width`.

```
/* Base: desktop largo */
body {
    font-size: 16px;
    display: flex;
    flex-direction: row;
    max-width: 1200px;
    margin: 0 auto;
}

/* Tablet e pi piccoli */
@media (max-width: 1199px) {
    body {
        max-width: 100%; /* togliamo il limite massimo */
        margin: 0;      /* eliminiamo i margini */
    }
}

/* Mobile */
```

```
@media (max-width: 767px) {  
  body {  
    flex-direction: column; /* impila gli elementi in  
      verticale */  
  }  
}
```